PRESENTACIÓN DE CAMBIOS

Una iteración para mejorar el diseño

UADE BEATS

Iteración 4

Patrones de diseño, documentación UML y verificación del comportamiento.

89
tipos documentados

19
diagramas PNG / SVG

103
comprobaciones aprobadas

Para exponer: El objetivo fue mejorar cómo está organizado y explicado el sistema.

¿De dónde partimos y qué cambió?

La base funcional proviene de la iteración 4.

Ya existía	Se trabajó en iteración 4
Aplicación Java con consola y Swing.	Revisión de responsabilidades y patrones.
SQLite y API HTTP para clientes remotos.	Strategy explícito para el alcance de eventos.
Tres roles y controles de permisos.	Diagramas y casos de uso trazados a los permisos.
Integración con calendarios Google y Apple.	Pruebas de regresión y documentación reproducible.

Para exponer: La API, los DTO y el esquema SQLite conservaron sus contratos.

GSD organizó el trabajo en pasos verificables

Primero se investigó; después se implementó y se contrastó el resultado.

1

Investigar

Código, UML y catálogo de patrones.

→

2

Planificar

Alcance, decisiones y criterios de aceptación.

→

3

Ejecutar

Refactorización y generación de diagramas.

→

4

Verificar

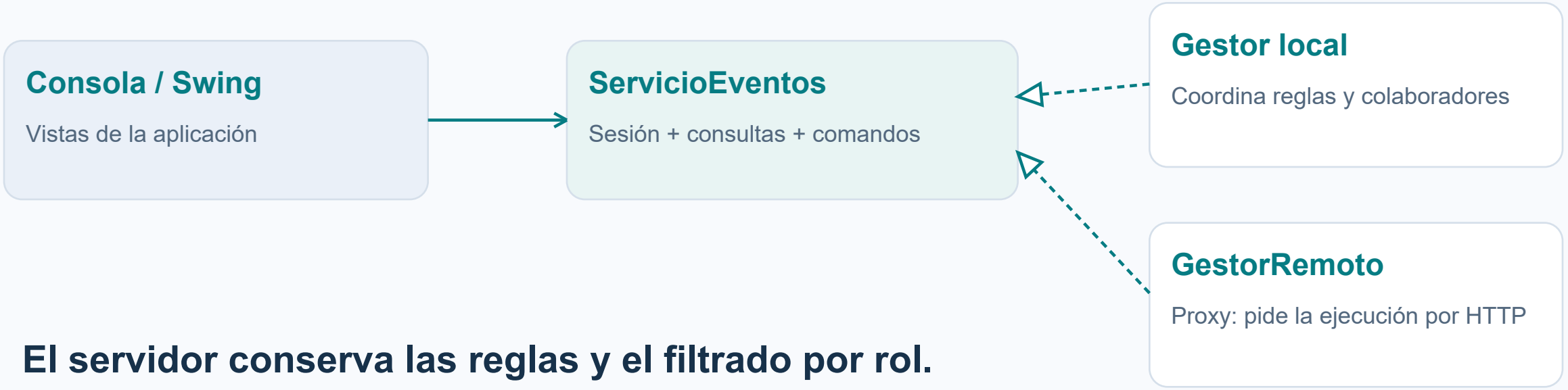
Pruebas, cobertura e inspección visual.

Quedaron planes, investigación, resúmenes y evidencia de verificación.

Para exponer: Cada decisión se vinculó con un problema real y una evidencia de validación.

La arquitectura mantiene un punto de entrada común

Consola y Swing usan el mismo contrato, tanto en modo local como remoto.



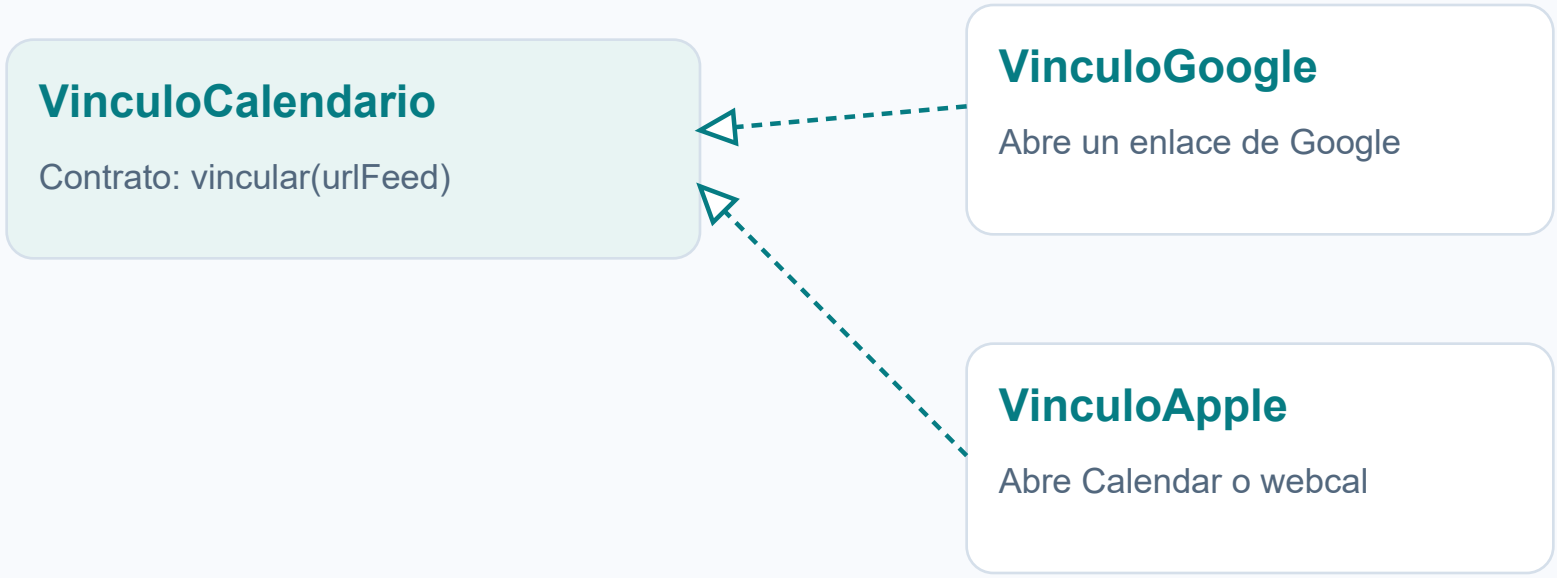
El servidor conserva las reglas y el filtrado por rol.

Las vistas no necesitan conocer las consultas SQL ni reconstruir las reglas del negocio.

Para exponer: Cambiar el modo de conexión no obliga a duplicar las pantallas.

Strategy ya existía en los calendarios

La tarea común es suscribir un evento; cambia el algoritmo de apertura del proveedor.



La vista elige

El gestor obtiene una URL autorizada e invoca la estrategia recibida.

En remoto, la apertura ocurre en el equipo cliente.

Para exponer: Strategy también puede recibirse como parámetro; no necesita guardarse en un atributo.

Strategy nuevo: separar el alcance de consulta

Se extrajo el algoritmo de visibilidad que estaba dentro del gestor.

Antes: decisión dentro del gestor

Un método comprobaba el rol y recorría los artistas del evento.

La regla era válida, pero estaba mezclada con la coordinación de casos de uso.

Ahora: delegación a políticas

El gestor pide una decisión a un contrato.

Dos algoritmos concretos expresan el catálogo general y la participación del artista.

```
return visibilidad.esVisible(actual, evento);
```

Para exponer: La regla observable se conserva; cambia el lugar donde se expresa y se prueba.

Cuatro tipos hacen explícita la colaboración

Dos algoritmos alcanzan para resolver los tres roles actuales.



Sin cuenta o sin evento: no hay visibilidad.

La participación se compara por ID persistente, incluso después de cargar los datos desde SQLite.

Para exponer: Se reutiliza el mismo alcance en listados, calendarios y controles de pertenencia.

Un ejemplo permite comprobar la regla

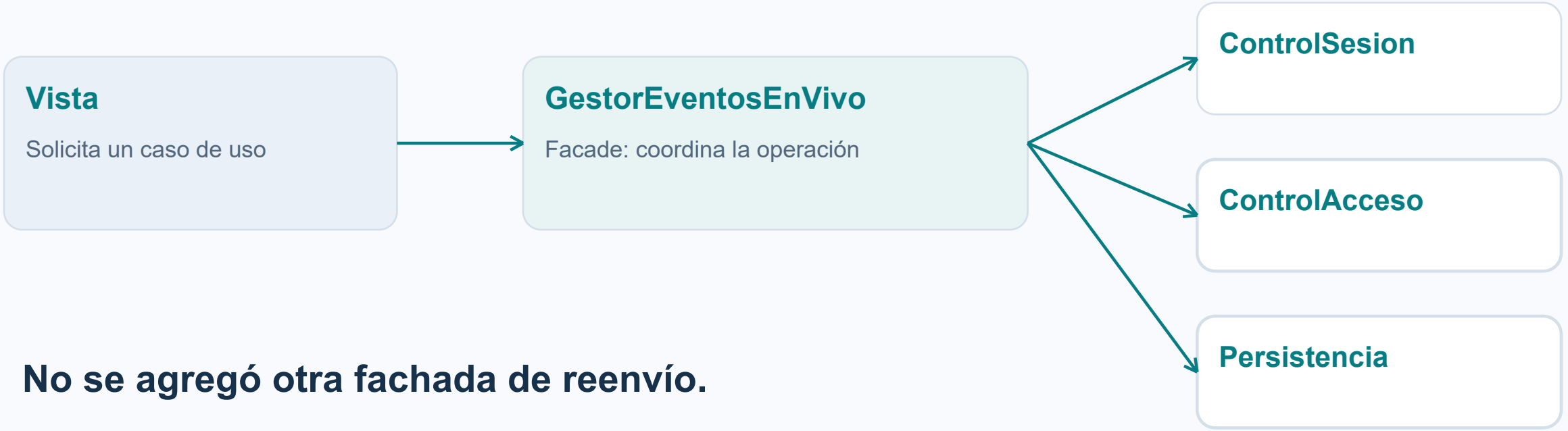
Supongamos que el artista 10 participa en el evento 100, pero no en el 101.

Cuenta / situación	Evento 100	Evento 101
Usuario autenticado	Visible	Visible
Administrador autenticado	Visible	Visible
Artista 10	Visible	No visible
Otra instancia del artista 10, mismo ID	Visible	No visible
Sin cuenta	No visible	No visible

Para exponer: La identidad se reconoce por ID y el algoritmo se elige por rol.

Facade ya estaba en el gestor

GestorEventosEnVivo simplifica el uso de varios colaboradores internos.



No se agregó otra fachada de reenvío.

ServicioEventos es el contrato; GestorRemoto es el Proxy para acceder al servidor.

Para exponer: El patrón se identifica por la responsabilidad que cumple, no solo por su nombre.

Composite y Decorator: cuándo aportan valor

Evaluar un patrón también implica decidir cuándo conviene posponerlo.

Composite

Uso real: paneles Swing que contienen otros componentes.

En el dominio: no existe un árbol de eventos.

Aplicación futura: festivales con escenarios y subeventos anidados.

Decorator

Situación actual: no hay envoltorios propios que agreguen capas al mismo contrato.

Candidato futuro: métricas de persistencia o notificaciones, si se necesitan.

Decisión: no agregar capas sin un requisito real.

Para exponer: Una lista no basta para ser Composite y un Proxy no equivale a Decorator.

UML pasó de vistas parciales a cobertura verificable

Se documentan clases, interfaces, enums, records y tipos anidados con nombre.

84	89	19
fuentes Java	tipos con nombre	diagramas entregados

Cada clase muestra atributos y operaciones con tipos y visibilidad.

Las relaciones distinguen herencia, realización, asociación, composición justificada y dependencia.

El inventario contrasta lo declarado en los fuentes con las clases compiladas. El SVG permite ampliar el modelo integral.

Para exponer: La generación automática controla cobertura; la semántica UML requiere revisión humana.

Una colección no implica propiedad del ciclo de vida

Extracto didáctico del modelo: se muestran relaciones reales y se omiten miembros por claridad.



El artista existe independientemente de un evento.

Para exponer: El diagrama integral se incorpora como anexo SVG para inspeccionarlo con zoom.

Los casos de uso distinguen a los tres actores

Las 20 acciones se contrastaron con Permisos.java; las reglas de rol se conservaron.

Usuario

Consulta eventos y capacidad.

Solicita ingreso a su nombre.

Ve sus registros y suscribe calendarios.

Artista

Crea y gestiona eventos donde participa.

Consulta sus conectados.

Suscribe sus eventos a calendarios.

Administrador

Administra cuentas y eventos.

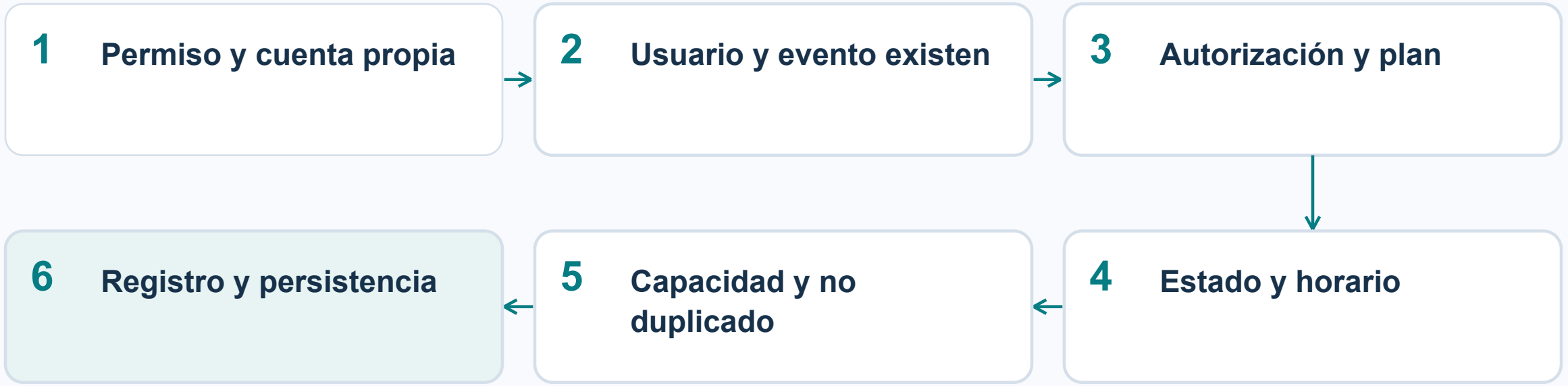
Activa, desactiva y expulsa usuarios.

Consulta todos los registros.

Para exponer: El administrador no hereda todos los casos del usuario: no tiene la acción Mis registros.

El flujo de ingreso permite explicar las decisiones

La autorización de la acción se comprueba antes de evaluar el acceso al recital.



Si una validación de acceso falla, se devuelve el motivo y se registra el rechazo.

Para exponer: Una falta de permiso se rechaza antes de procesar el ingreso; no es un acceso fallido.

Los flujos de gestión también respetan el alcance

La misma política se reutiliza después de verificar el permiso de cada operación.

Artista: gestionar un evento

1. Selecciona un evento propio.

2. Se valida su participación y la transición de estado.

3. Se guarda y se refresca la vista.

Evento ajeno: la operación se deniega.

Administrador: desactivar un usuario

1. Se comprueba el permiso.

2. Se impide desactivar la propia cuenta.

3. Si desactiva, desconecta al usuario y registra expulsiones.

4. Se guardan los cambios.

Para exponer: Ver un evento y estar autorizado a modificarlo son controles relacionados, pero distintos.

La evidencia de la iteración: 103 comprobaciones

Resultados registrados en la ejecución previa de la iteración, no una nueva auditoría integral.

Área	Qué se comprobó
Políticas y permisos	Roles, ID persistente, nulos, eventos propios y ajenos.
Servicio local y HTTP	Misma regla al listar, obtener calendario y cambiar estado.
Persistencia SQLite	Guardar y cargar conserva asociaciones y visibilidad.
Documentación UML	89 tipos con miembros; 20 acciones; 19 pares PNG/SVG.

Para exponer: Las pruebas de regresión muestran que la refactorización conserva el comportamiento esperado.

Qué quedó fuera y qué conviene continuar

Distinguir evidencia de funcionamiento y tareas pendientes ayuda a defender el trabajo.

Límites de las pruebas

No se confirmó una suscripción real en Google o Apple.

No se automatizaron todos los clics de Swing.

La inspección UML no es una certificación formal.

Mejoras identificadas

Definir vencimiento de sesiones.

Fortalecer validación de datos y URL pública.

Revisar consistencia entre memoria y base si falla un guardado.

Para exponer: Un rollback SQL no revierte automáticamente los objetos que ya cambiaron en memoria.

Aporte principal y preguntas para la defensa

La iteración mejora separación, explicación y verificabilidad del diseño.

Una regla explícita, una documentación trazable y evidencia para sostener las decisiones.

Pregunta	Respuesta breve
¿Por qué Strategy?	Separa dos algoritmos de alcance y permite probarlos sin UI.
¿Por qué no aplicar todos los patrones?	Cada abstracción debe resolver una necesidad concreta.
¿Cambió lo que puede hacer cada rol?	No. Se preservaron los permisos y se probaron sus resultados.

Para exponer: La calidad se muestra en la justificación y en las pruebas, no en la cantidad de patrones.

ANEXO / PARA CONSULTAR

Material de respaldo y referencias

Los diagramas detallados se incluyen como archivos adjuntos dentro de este PDF.

Adjunto técnico: diagrama completo de clases de esta entrega, en SVG ampliable.

En un lector compatible, abrir el panel de archivos adjuntos. Algunos visores web no muestran este panel.

UML 2.5.1 - OMG

Notación y generación - PlantUML

Strategy, Facade, Composite y Decorator

Jerarquía de componentes - Java SE 21

Base documental: PATRONES.md, REVISION-UML.md, CASOS-DE-USO.md y VERIFICACION.md, en docs/iteracion-4/.

Para exponer: Las figuras didácticas resumen el modelo; los anexos conservan el nivel de detalle técnico.